LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL IV & V
PERULANGAN & PERCABANGAN

Disusun Oleh:

Alwisyah Putra 105224052

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

DAFTAR ISI

I. Pendahuluan.....2

II. Variabel..... 2

III. Constructor & Method..... 2

IV. Dokumentasi dan Pembahasan Code.....2

V. Kesimpulan.....2

VI. Daftar Pustaka.....2

I. Pendahuluan

Pada tugas take home ini, saya akan membangun sebuah program simulasi sistem reservasi kamar hotel berbasis objek dengan menerapkan prinsip encapsulation secara ketat menggunakan bahasa pemrograman Java. Program utama yang akan dikembangkan adalah kelas KamarHotel yang berfungsi sebagai blueprint untuk mengelola data kamar, seperti nomor kamar, tipe kamar, kapasitas maksimal, harga per malam, serta status ketersediaan. Tantangan utama dari studi kasus ini terletak pada implementasi validasi bisnis yang ketat pada metode setter, seperti memaksa tipe kamar menjadi "Reguler" jika input tidak sesuai pilihan yang ditentukan, serta memastikan harga per malam tidak kurang dari Rp50.000. Selain itu, sistem juga harus mampu menangani pemesanan dengan metode overloading yang memvalidasi jumlah tamu agar tidak melebihi kapasitas kamar, serta menerapkan logika diskon pada perhitungan total bayar berdasarkan minimal masa menginap dan penggunaan kode voucher "PROMO". Dengan begitu, program ini dirancang untuk meminimalisir kesalahan operasional yang sering terjadi, seperti kelebihan muatan kamar hingga penyalahgunaan kode voucher, sekaligus menguji ketahanan sistem terhadap input yang tidak valid.

II. Variabel

1. KamarHotel.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	nomorKamar	String	Menyimpan nomor identifikasi kamar hotel
2.	tipeKamar	String	Menyimpan tipe kamar (Reguler, Premium, Suite)
3.	kapasitasMaksimal	int	Menyimpan jumlah maksimal tamu yang dapat menginap

4.	hargaPerMalam	double	Menyimpan harga sewa kamar per malam
5.	isTersedia	boolean	Menyimpan status ketersediaan kamar (true = tersedia, false = terisi)

2. App.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	kamar1	KamarHotel	Objek kamar hotel pertama untuk testing
2.	kamar2	KamarHotel	Objek kamar hotel kedua untuk testing
3.	args	String[]	Parameter command line arguments

III. Constructor & Method

1. KamarHotel.java

No.	Nama Method	Jenis	Fungsi
1.	KamarHotel(String, String, int)	Instance Method	Menginisialisasi objek kamar dengan 3 parameter, harga default 0.0, status tersedia true
2.	KamarHotel(String, String, int, double)		Menginisialisasi objek kamar dengan 4 parameter lengkap, status tersedia true
3.	getNomorKamar()		Mengembalikan nilai nomor kamar
4.	getTipeKamar()		Mengembalikan nilai tipe

			kamar
5.	getKapasitasMaksimal()		Mengembalikan nilai kapasitas maksimal
6.	getHargaPerMalam()		Mengembalikan nilai harga per malam
7.	isTersedia()		Mengembalikan status ketersediaan kamar
8.	setTipeKamar(String)		Mengatur tipe kamar dengan validasi (hanya Reguler, Premium, Suite), default ke Reguler jika invalid
9.	setHargaPerMalam(double)		Mengatur harga dengan validasi minimal Rp50.000, auto-set ke 50000 jika di bawah
10.	pesanKamar()		Memesan kamar tanpa parameter jumlah tamu, ubah status jika tersedia
11.	pesanKamar(int)		Memesan kamar dengan validasi kapasitas tamu, cek ketersediaan
12.	batalPesan()		Membatalkan pemesanan, mengubah status kamar menjadi tersedia
13.	hitungTotalBayar(int)		Menghitung total tagihan berdasarkan jumlah malam
14.	hitungTotalBayar(int,		Menghitung total dengan

	String)		diskon 20% jika voucher "PROMO" dan minimal 3 malam
15.	cetakInfoKamar()		Menampilkan informasi lengkap kamar ke console

2. App.java

No.	Nama Method	Jenis	Fungsi
1.	main(String[])	static method	Dideklarasikan dengan static, dipanggil JVM tanpa membuat objek App

IV. Dokumentasi dan Pembahasan Code

1. File KamarHotel.java

```java
class KamarHotel {

    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;

    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0.0;
        this.isTersedia = true;
    }

    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(hargaPerMalam);
        this.isTersedia = true;
    }
```

saya membuat kelas KamarHotel di file terpisah yang memiliki lima atribut private sebagai penerapan enkapsulasi. Kemudian saya membuat dua buah konstruktor. konstruktor pertama menerima tiga parameter (nomorKamar, tipeKamar, kapasitasMaksimal) yang secara otomatis mengatur hargaPerMalam = 0 dan isTersedia = true, sementara konstruktor kedua menerima empat parameter (nomor,

tipe, kapasitas, dan harga) dengan status ketersediaan juga langsung diset true. Pada kedua konstruktor tersebut, metodh setTipeKamar() saya panggil untuk memastikan validasi tipe kamar, dan pada konstruktor kedua juga dipanggil setHargaPerMalam() untuk memvalidasi harga.

```java
public String getNomorKamar() {
    return nomorKamar;
}

public String getTipeKamar() {
    return tipeKamar;
}

public int getKapasitasMaksimal() {
    return kapasitasMaksimal;
}

public double getHargaPerMalam() {
    return hargaPerMalam;
}

public boolean isTersedia() {
    return isTersedia;
}
```

kemudian saya buat Getter untuk seluruh atribut pada kelas KamarHotel, yaitu getNomorKamar(), getTipeKamar(), getKapasitasMaksimal(), getHargaPerMalam(), dan isTersedia(). Alur kerja dari method ini hanya mengembalikan (return) nilai dari atribut privat yang sesuai tanpa melakukan proses validasi atau modifikasi apapun. Dengan adanya getter ini, kelas lain (seperti kelas Main) tetap dapat mengakses

nilai-nilai atribut kamar secara tidak langsung, meskipun atribut tersebut dideklarasikan sebagai private.

```java
public void setTipeKamar(String tipeKamar) {
        if (tipeKamar.equalsIgnoreCase("Reguler") ||
                tipeKamar.equalsIgnoreCase("Premium") ||
                tipeKamar.equalsIgnoreCase("Suite")) {

            if (tipeKamar.equalsIgnoreCase("Reguler")) {
                this.tipeKamar = "Reguler";
            } else if (tipeKamar.equalsIgnoreCase("Premium")) {
                this.tipeKamar = "Premium";
            } else {
                this.tipeKamar = "Suite";
            }

        } else {
            System.out.println("PERINGATAN: Tipe kamar '" + tipeKamar +
                    "' tidak valid! Harus 'Reguler', 'Premium', atau 'Suite'.");
            System.out.println("Sistem otomatis mengubah ke 'Reguler'.");
            this.tipeKamar = "Reguler";
        }
    }

    public void setHargaPerMalam(double hargaPerMalam) {
        if (hargaPerMalam >= 50000) {
            this.hargaPerMalam = hargaPerMalam;
        } else {
            System.out.println("PERINGATAN: Harga Rp" + hargaPerMalam + " terlalu rendah WOY! Minimal Rp50000.");
            System.out.println("Sistem otomatis mengubah harga ke Rp50000.");
            this.hargaPerMalam = 50000.0;
        }
    }
```

Setelah itu saya membuat dua setter dengan parameter setTipeKamar variabel tipeKamar dan setHargaPerMalam variabel hargaPerMalam. Pada metodh setTipeKamar(), alurnya dimulai dengan memeriksa apakah input parameter tipeKamar cocok dengan salah satu dari tiga nilai yang diperbolehkan yaitu "Reguler", "Premium", atau "Suite" menggunakan metode equalsIgnoreCase() agar tidak membedakan huruf besar-kecil. Jika input cocok dengan "Reguler", maka atribut this.tipeKamar diset menjadi "Reguler"; jika cocok dengan "Premium" maka diset "Premium"; selain itu (fallback-nya) otomatis diset menjadi "Suite". Setelah bagian validasi sukses, terdapat blok else yang akan dieksekusi jika input tidak cocok dengan ketiga pilihan, di mana sistem akan mencetak peringatan bahwa tipe kamar tidak valid dan secara otomatis memaksa nilai atribut menjadi "Reguler".

Pada metode setHargaPerMalam(), memeriksa apakah hargaPerMalam yang diinput bernilai lebih besar atau sama dengan 50000. Jika kondisi tersebut terpenuhi, maka nilai atribut this.hargaPerMalam langsung diisi dengan nilai input. Sebaliknya, jika harga di bawah 50000, sistem akan mencetak peringatan ke layar (dengan pesan yang

agak santai seperti "WOY") lalu memaksa nilai atribut menjadi 50000.0.

```java
public void pesanKamar() {
        if (isTersedia) {
            isTersedia = false;
            System.out.println("Pemesanan berhasil YEYYY! Kamar " + nomorKamar + " telah dibooking.");
        } else {
            System.out.println("GAGAL!!!!!! Kamar " + nomorKamar + " sudah terisi");
        }
    }

    public void pesanKamar(int jumlahTamu) {
        if (!isTersedia) {
            System.out.println("GAGAL: Kamar " + nomorKamar + " sudah terisi.");
            return;
        }

        if (jumlahTamu > kapasitasMaksimal) {
            System.out.println("GAGAL: Jumlah tamu (" + jumlahTamu + " orang) melebihi kapasitas maksimal ("
                    + kapasitasMaksimal + " orang)!");
            System.out.println("SISTEM MENOLAK PESANAN - OVERCAPACITY WEEEEEEEEE!!!!");
            return;
        }

        isTersedia = false;
        System.out.println("Pemesanan berhasil! Kamar " + nomorKamar +
                " untuk " + jumlahTamu + " tamu telah dipesan.");
    }

    public void batalPesan() {
        if (!isTersedia) {
            isTersedia = true;
            System.out.println("Pembatalan berhasil! Kamar " + nomorKamar + " kini Jadi kosong lagi");
        } else {
            System.out.println("Kamar " + nomorKamar + " belum dipesan.");
        }
    }
```

Selanjutnya saya membuat method pesanKamar() serta metode batalPesan() untuk mengelola status pemesanan kamar. Pada metode pesanKamar() tanpa parameter, dimulai dengan mengecek apakah kamar sedang tersedia (!isTersedia artinya jika isTersedia == true maka kondisi bernilai true). jika negasi dari status isTersedia yang berarti "jika kamar tidak tersedia", maka akan menampilkan peringatan kamar sudah penuh. Setelah itu, jika kondisi tidak terpenuhi, sistem akan mengubah isTersedia = false dan mencetak pesan sukses.

Pada metodh pesanKamar(int jumlahTamu), pertama, metode mengecek apakah kamar sudah tidak tersedia (!isTersedia). Jika sudah terisi, langsung cetak peringatan dan hentikan proses dengan return. Jika kamar masih tersedia, metode kemudian memeriksa apakah jumlahTamu melebihi kapasitasMaksimal. Jika iya, sistem mencetak pesan penolakan karena overcapacity dan menghentikan pemesanan tanpa mengubah status kamar. Jika kedua pengecekan lolos (kamar tersedia dan jumlah tamu sesuai kapasitas), maka isTersedia diubah menjadi false dan mencetak pesan sukses.

Ketiga, metode batalPesan() bekerja dengan mengecek apakah kamar sedang tidak tersedia (!isTersedia), yang berarti sedang dalam keadaan terisi. Jika benar, maka status dikembalikan menjadi true (tersedia lagi) dan mencetak pesan sukses pembatalan. Jika kamar ternyata sudah dalam keadaan tersedia (else), sistem mencetak informasi bahwa kamar belum dipesan, sehingga tidak perlu dibatalkan.

```java
    public double hitungTotalBayar(int jumlahMalam) {
        double total = hargaPerMalam * jumlahMalam;
        System.out.println("Total tagihan (" + jumlahMalam + " malam): Rp" + total);
        return total;
    }

    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
        double total = hargaPerMalam * jumlahMalam;

        boolean isVoucherValid = kodeVoucher.equals("PROMO");
        boolean isMinMalamMet = jumlahMalam >= 3;

        if (isVoucherValid && isMinMalamMet) {
            double diskon = total * 0.20;
            double totalSetelahDiskon = total - diskon;

            System.out.println("Voucher PROMO berhasil digunakan!");
            System.out.println("Harga normal: Rp" + total);
            System.out.println("Diskon 20%  : Rp" + diskon);
            System.out.println("Total bayar : Rp" + totalSetelahDiskon);

            return totalSetelahDiskon;

        } else {
            System.out.println("Voucher '" + kodeVoucher + "' gagal digunakan:");

            if (!isVoucherValid) {
                System.out.println("Kode voucher tidak valid");
            }
            if (!isMinMalamMet) {
                System.out.println("Minimal menginap 3 malam sedangkan (Anda hanya menginap: " + jumlahMalam + " malam)");
            }

            System.out.println("Total tagihan anda (belum diskon): Rp" + total);
            return total;
        }
    }

    public void cetakInfoKamar() {
        System.out.println("----------------------------------------");
        System.out.println("INFORMASI KAMAR HOTEL PERTAMINA SIMPRUG");
        System.out.println("----------------------------------------");
        System.out.println("Nomor Kamar      : " + nomorKamar);
        System.out.println("Tipe Kamar       : " + tipeKamar);
        System.out.println("Kapasitas Maks   : " + kapasitasMaksimal + " orang");
        System.out.println("Harga/Malam      : Rp" + hargaPerMalam);

        String status = isTersedia ? "TERSEDIA" : "TERISI";
        System.out.println("Status           : " + status);
        System.out.println("----------------------------------------");
    }
}
```

Pada bagian terakhir saya membuat method hitungTotalBayar() serta metode cetakInfoKamar() untuk keperluan penagihan dan pencetakan informasi kamar.

Pada metode hitungTotalBayar(int jumlahMalam) tanpa kode voucher, metode menghitung total tagihan dengan rumus hargaPerMalam * jumlahMalam, lalu mencetak hasilnya ke layar, dan mengembalikan nilai total tersebut sebagai double.

Pada metode hitungTotalBayar(int jumlahMalam, String kodeVoucher). Pertama, metode menghitung total standar dengan rumus yang sama. Kemudian, sistem membuat dua variabel boolean isVoucherValid untuk mengecek apakah kode voucher yang dimasukkan sama dengan "PROMO" (menggunakan equals(), bukan equalsIgnoreCase()), dan isMinMalamMet untuk mengecek apakah jumlahMalam lebih besar atau sama dengan 3. Setelah itu, terdapat percabangan if-else: jika kedua kondisi terpenuhi (voucher valid dan minimal 3 malam), maka sistem menghitung diskon 20% (total * 0.20), menghitung total setelah diskon, mencetak rincian tagihan lengkap (harga normal, diskon, total bayar), lalu mengembalikan nilai total setelah diskon. Sebaliknya, jika salah satu atau kedua kondisi tidak terpenuhi, sistem akan mencetak pesan bahwa voucher gagal digunakan, lalu mengecek secara terpisah penyebab kegagalannya (apakah karena kode voucher salah, atau karena masa menginap kurang dari 3 malam, atau keduanya), kemudian mencetak total tagihan normal dan mengembalikan nilai total standar tanpa diskon.

Terakhir, metode cetakInfoKamar() berfungsi untuk menampilkan seluruh informasi atribut kamar secara rapi ke layar. dimulai dengan mencetak garis pemisah, judul "INFORMASI KAMAR HOTEL PERTAMINA SIMPRUG", lalu mencetak nilai dari nomorKamar, tipeKamar, kapasitasMaksimal, dan hargaPerMalam. Untuk status ketersediaan, saya menggunakan operator ternary ? : yang mengubah nilai boolean isTersedia menjadi string "TERSEDIA" jika true, atau "TERISI" jika false. Kemudian diakhiri dengan garis pemisah lagi.

2. File App.java

```java
public class App {
    public static void main(String[] args) throws Exception {
        KamarHotel kamar1 = new KamarHotel("5206", "Deluxe", 3);

        System.out.println("=== KAMAR 1 (SEBELUM VALIDASI) ===");
        kamar1.cetakInfoKamar();

        System.out.println("\n=== UJI SETTER KAMAR 1 ===");
        kamar1.setTipeKamar("Presidential");
        kamar1.setHargaPerMalam(-10000);

        System.out.println("\n=== KAMAR 1 (SETELAH VALIDASI) ===");
        kamar1.cetakInfoKamar();

        KamarHotel kamar2 = new KamarHotel("2805", "Premium", 2, 750000);

        System.out.println("\n=== KAMAR 2 ===");
        kamar2.cetakInfoKamar();

        System.out.println("\n=== UJI PEMESANAN KAMAR 2 (KAPASITAS 2) ===");
        kamar2.pesanKamar(4);
        kamar2.pesanKamar(2);
        kamar2.pesanKamar();

        System.out.println("\n=== PERHITUNGAN TAGIHAN ===");
        kamar1.hitungTotalBayar(2, "PROMO");
        kamar2.hitungTotalBayar(4, "PROMO");

        System.out.println("\n=== STRUK AKHIR ===");
        kamar1.cetakInfoKamar();
        kamar2.cetakInfoKamar();
    }
}
```

di file terpisah saya membuat file app.java dengan kelas App yang berisi metodh main() sebagai entry point program. dimulai dengan membuat objek kamar1 menggunakan konstruktor pertama (pendaftaran kilat) dengan parameter nomor kamar "5206", tipe "Deluxe", dan kapasitas 3 orang. Program kemudian mencetak informasi awal kamar1 sebelum validasi, di mana tipe "Deluxe" sebenarnya tidak valid karena hanya menerima "Reguler", "Premium", atau "Suite", akibatnya saat konstruktor memanggil setTipeKamar(), tipe akan dipaksa menjadi "Reguler". Selanjutnya, sayamelakukan uji ketahanan setter dengan mencoba mengubah tipe kamar menjadi "Presidential" (tidak valid) dan harga menjadi -10000 (di bawah minimum). Sistem diharapkan menolak kedua input tersebut, memaksa tipe menjadi "Reguler" dan harga menjadi Rp50.000, lalu mencetak peringatan ke layar. Setelah itu, informasi kamar1 dicetak ulang untuk menunjukkan hasil akhir setelah validasi.

Kemudian saya membuat objek kamar2 menggunakan konstruktor kedua (pendaftaran lengkap) dengan nomor "2805", tipe "Premium", kapasitas 2 orang, dan harga Rp750.000, lalu mencetak informasinya. Selanjutnya dilakukan uji pemesanan pada kamar2: pertama mencoba pesanKamar(4) yang seharusnya ditolak karena melebihi kapasitas maksimal 2 orang; kemudian pesanKamar(2) yang seharusnya berhasil karena jumlah tamu sesuai kapasitas dan kamar masih tersedia; lalu pesanKamar() tanpa parameter pada kamar yang sama yang seharusnya gagal karena kamar sudah berstatus terisi.

Setelah itu saya melakukan uji perhitungan tagihan. pada kamar1 dihitung total bayar untuk 2 malam dengan voucher "PROMO" , akan gagal mendapat diskon karena masa menginap kurang dari 3 malam; pada kamar2 dihitung total bayar untuk 4 malam dengan voucher "PROMO" , hasilnya berhasil mendapat potongan 20%. Terakhir, program mencetak struk informasi akhir dari kedua objek kamar untuk menampilkan seluruh kondisi akhir atribut setelah semua proses selesai.

V. Kesimpulan

Berdasarkan praktikum yang telah dilakukan pada Modul IV dan V yang berjudul , dapat disimpulkan bahwa program sistem reservasi kamar hotel yang dibangun telah berhasil menerapkan prinsip-prinsip Pemrograman Berorientasi Objek (OOP) secara tepat. Penerapan enkapsulasi yang saya lakukan dibuktikan dengan penggunaan access modifier private pada seluruh atribut kelas KamarHotel, sehingga data tidak dapat diakses langsung dari luar kelas dan hanya dapat dimanipulasi melalui metode publik (getter dan setter) yang telah disediakan. Validasi bisnis yang ketat pada metodh setTipeKamar() dan setHargaPerMalam() terbukti mampu menangkal input tidak valid seperti tipe "Presidential" dan harga negatif, secara otomatis memaksa nilai menjadi "Reguler" dan Rp50.000 sesuai aturan. Selain itu, implementasi methodh pesanKamar() dan hitungTotalBayar() memungkinkan sistem merespon secara berbeda tergantung jumlah parameter yang diberikan, seperti validasi kapasitas tamu pada pemesanan dan pemberian diskon 20% dengan syarat minimum 3 malam serta kode voucher "PROMO". Hasil pengujian di kelas App menunjukkan bahwa kedua

objek kamar (kamar1 dan kamar2) berhasil memblokir pemesanan overcapacity, menolak penggunaan voucher yang tidak memenuhi syarat, serta mencetak struk informasi akhir dengan rapi. Dengan demikian menurut saya, program ini telah memenuhi seluruh spesifikasi yang diminta pada Take Home Task Modul 4 & 5 dan layak digunakan untuk meminimalisir kesalahan operasional pada jaringan manajemen perhotelan.

VI. Daftar Pustaka

Timotheus Ferdinand Tjondrojo and Yan Stephen Christian Immanuel Hutagalung (2026a) **Modul 4 - The Core of OOP.** Laboratorium Komputer, Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.

Timotheus Ferdinand Tjondrojo and Yan Stephen Christian Immanuel Hutagalung (2026b) **Modul 5 - Encapsulation.** Laboratorium Komputer, Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.

THT Modul 4 & 5 (2026) Take Home Task Modul 4 dan 5, Universitas Pertamina.